

UNIVERSIDAD DE MONTERREY

Escuela de Ingeniería y Tecnología

Ingeniería en Tecnologías Computacionales

Evaluación Arquitectónica

Autor(es)

600046 Fernando Olivares Del Valle

Profesor (a)

Prof. Raúl Morales Salcedo

Doy mi palabra que he realizado esta actividad con integridad académica

San Pedro Garza García, N.L. 27 de Agosto, 2026

Evaluación Arquitectónica:

Respecto al sistema desarrollado a lo largo de este ejercicio, es relevante mencionar la arquitectura monolítica server-side utilizada, donde es importante señalar si fue correcto tomar esta decisión de la arquitectura para el escenario propuesto, comparándola con otras soluciones como desacoplada por componentes y con microservicios. Para esto es importante tomar en cuenta la complejidad que puede representar una arquitectura u otra, tanto operativamente, como en el despliegue, la escalabilidad, el mantenimiento, la seguridad y el tamaño del equipo de desarrollo.

En este caso se propuso desarrollar un sistema para una librería, con un equipo de un solo desarrollador y un volumen limitado de datos y tráfico, el sistema monolítico representa la opción más adecuada para este ejercicio. La complejidad operativa era mínima, puesto que no se tenía que pensar en soluciones que involucren múltiples servicios de terceros, además no existía comunicación entre procesos que pudiera afectar al otro si uno fallaba y todo el sistema se encontraba en un solo repositorio.

Respecto al despliegue de la aplicación, este se reducía a la instalación de Node.js, la configuración de PostgreSQL y publicar la aplicación en un servidor. Esta simplicidad hace que el desarrollo de este proyecto se enfocara principalmente en el aprendizaje.

Analizando la otra alternativa, una solución desacoplada por componentes utilizando una API, esta empieza a tomar mayor relevancia si el objetivo del sistema es que exista, además de una aplicación web, una aplicación móvil o si existen distintos equipos de trabajo en donde se encargan de cosas distintas y de forma separada, como el frontend y el backend, esto con el propósito de que no exista ningún bloqueo entre ellos. Sin embargo, esto representa que ahora haya dos sistemas que desplegar, e implica llevar más documentación, versionarlos y mantenerlos sincronizados, lo cual para un solo desarrollador significa mucho trabajo y tiempo.

De forma más detallada, en la solución desacoplada por componentes, los microservicios se encargan de que, en vez de tener un solo backend único, se dividan las responsabilidades en servicios independientes, en donde cada uno tiene su base de datos (usuarios, libros, imágenes). Esto sirve mucho cuando el tráfico de la aplicación web o el equipo de desarrollo crecen lo suficiente, que escalar el sistema no es opción, sino que se requiere. El costo de hacer esta modificación dentro del sistema representa que se tiene que mantener más infraestructura, la comunicación entre servicios puede llegar a fallar o volverse lenta. Para un proyecto como una librería con un volumen de datos y tráfico como el de este ejercicio, que es puramente educativo, una arquitectura de microservicios representaría una carga innecesaria.

Otro aspecto es el mantenimiento del sistema, el monolito al estar organizado por capas ya tiene una buena separación entre las capas, haciendo que cada una de ellas tenga su propia responsabilidad clara y al querer modificar una de ellas no debería afectar a las demás. Al crecer a una desacoplada, es más fácil el mantenimiento puesto que se evita que un cambio de una persona bloquee el trabajo de otra.

Respecto a la seguridad, el monolito tiene una gran ventaja puesto que su patrón de diseño por capas hace que, al ser atacado, sea fácil identificar el punto de entrada, puesto que solo existe uno y un solo usuario privilegiado. Una arquitectura con microservicios representa un riesgo de seguridad porque cada servicio expuesto es un punto de entrada que necesita una capa de autenticación.

A manera de conclusión, el monolito con server-side es la decisión correcta para este tipo de escenarios educativos donde el tráfico y el equipo de desarrollo no son grandes. Lo único que cambiaría la decisión será que se tenga un mayor tráfico, se necesiten más recursos o se tenga un equipo de trabajo más grande.